Appello 03 marzo 2015



Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h45'.

Esercizio 1:
Esercizio 2:
Esercizio 3:
Esercizio 4:

Esercizio 1

Si consideri la classe `Poligonale` classe mutabile che rappresenta una sequenza (senza duplicazioni) di punti nello spazio tridimensionale. Ogni punto è un elemento di una classe Punto, accennata in seguito. La classe `Poligonale` è così specificata:

```
public class Poligonale {

 public Poligonale();//costruisce una poligonale vuota.

 //@ensures /result e' la cardinalita' dell'insieme
 public int size() {...}

 //@ensures /result e' il punto i-esimo della sequenza, con 0<=i<size().
 //@signals IndexOutOfBoundException e) (i<0 || i>=size();
 public Punto get(int i) throws IndexOutOfBoundException {...}

 //@ensures (*inserisce un nuovo punto al termine della sequenza)
 //@signals DuplicateException e) (*eccezione se p esiste in this*)
 public void inserisci(Punto p) throws IndexOutOfBoundException {...}

 //@ensures /result e' un iteratore a tutti i punti, nell'ordine della sequenza,
 //@ senza ripetizioni.
 public Iterator<Punto> punti() {...}

 //@ensures /result e' la lunghezza della linea spezzata che unisce,
 //@in ordine, i punti della Poligonale.
 public double lunghezza() {...}

......
}
```

La classe immutabile `Punto` ha un costruttore `Punto(double x, double y, double z)` e gli osservatori: `double getX(),double getY(), double getZ()`, di significato ovvio. La classe fornisce anche un metodo `distanza(Punto p)`, che calcola la distanza euclidea fra p e this. Infine, la classe `Punto` definisce `equals` in modo che due punti siano equals se, e solo se, hanno identiche coordinate.

a) Si scriva la specifica JML dei metodi `inserisci(Punto)`, `lunghezza()`.

b) Si definisca un opportuno invariante pubblico della classe `Poligonale`, in base alla descrizione che è stata data della classe stessa, e si argomenti se esso è verificato oppure no dalla specifica data.

c) Si consideri la seguente implementazione parziale di `Poligonale`:

```
public class Poligonale {
   private class Nodo{
     Punto p;
     Nodo n;
     Nodo(Punto p, Nodo n) {
          this.p = p;
          this.n=n;
     }
   }
   private Nodo testa;
   private int len; //rep
   public Poligonale() {
    testa = null;
    len=1;
  }
   public void inserisci(Punto p) throws DuplicateException {
     ----
     ----
     ----
     testa= new Nodo(p,testa);
     len++;
   }
   public int size() {
    ----
    ----
  }
}
```

c1) Completare il codice di `inserisci` dove sono presenti i trattini `----`.

c2) Scrivere l'invariante rappresentazione della classe.

c3) Definire la funzione di astrazione della classe `Poligonale`, tramite un opportuno private invariant o un'implementazione del metodo toString().

c4) Si implementi il metodo iteratore `punti()`, definendo le classi opportune.

Esercizio 2

Si consideri una variante di `Poligonale`, detta `PoligonaleBis`, che ha anche un metodo che consente di inserire un punto come successivo a un punto dato.

```
//@ensures (*inserisce un nuovo punto daInserire dopo p*)
 //@signals DuplicateException e) (*eccezione se p esiste in this*);
 public void inserisciDopo(Punto p,Punto daInserire) throws IndexOutOfBoundException
```

1. Specificare il metodo dato in JML.

2. E' possibile definire questa classe `PoligonaleBis` come derivata da`Poligonale`, rispettando il principio di sostituzione? Motivare la risposta.

3. Si consideri un'altra variante di `Poligonale`, detta `PoligonaleTer`, che ha la stessa definizione di `Poligonale`, ma il cui metodo inserisci e' cosi' descritto:

   ```
   //@ensures (*inserisce un nuovo punto in una posizione qualunque*);
   //@signals DuplicateException e) (*eccezione se p esiste in this*);
   public void inserisci(Punto p) throws IndexOutOfBoundException {...}
   ```

   E' possibile definire questa classe `PoligonaleTer` come derivata da `Poligonale`, rispettando il principio di sostituzione? E viceversa, Poligonale come derivata da `PoligonaleTer`? Motivare la risposta.

Esercizio 3

Si considerino le seguenti classi Java:

```
public class Frutto {
   public String toString(){
      return "Frutto";
   }
   public void macedonia(Frutto f){
      System.out.println("Frutto " + f);
   }
}

public class Mela extends Frutto {
   public String toString(){
      return "Mela";
   }
   public void macedonia(Frutto f){
      System.out.println("Frutto (Mela) " + f);
   }
   public void macedonia(Mela f){
      System.out.println("Mela " + f);
   }
}

public class Arancia extends Frutto {
   public String toString(){
      return "Arancia";
   }
   public void macedonia(Frutto f){
      System.out.println("Frutto (Arancia) " + f);
   }
   public void macedonia(Arancia f){
      System.out.println("Arancia " + f);
   }
}
```

e si spieghi cosa stamperebbe il seguente metodo `main`, motivando brevemente le risposte:

```
public class Test {
   public static void main(String args[]) {
      Frutto f1, f2;
      Mela m;
      Arancia a;
      f1 = new Frutto();
      f2 = new Arancia();
      m = new Mela();
      a = new Arancia();
      f1.macedonia(f1);
      f1.macedonia(f2);
      f2 = m;
      f1 = m;
      f1.macedonia(f2);
      a.macedonia(m);
      f1.macedonia(a);
   }
}
```

Esercizio 4

Si consideri il seguente frammento di programma:

```
0. numProdotti=0;
1. while (numProdotti <= PROD_MAX) {
2.     ----
3.     if (costoProdotto <= restaDaSpendere) {
4.        ----
          restaDaSpendere = restaDaSpendere - costoProdotto;
5.        if (restaDaSpendere > 0) {
6.               numProdotti++;
7.        } else {
8.               System.out.println("Soldi finiti!");
9.               break;
10.        }
11.    }
12. };
```

1. Si definiscano casi di test che coprano tutte le istruzioni (si considerino le parti tralasciate `----` come se fossero una singola macro-istruzione che non modifica alcuna delle variabili che appaiono nel frammento);

2. Se il criterio di copertura fosse quello di copertura dei branch si otterrebbe lo stesso risultato? Perché sì? Perché no?

3. Si calcoli la condizione logica che impone che il ciclo venga eseguito due volte, ma in modo che alla seconda iterazione si esca con l'istruzione break.